

Proposed ROS 2 file structure (new vs modified)

```
multirobot_ws/src/multirobot_nav/  
├─ package.xml                # MOD: add exec_depends  
├─ setup.py                  # MOD: add console_script for map_merger  
├─ setup.cfg                 # (unchanged)  
├─ multirobot_nav/  
│   ├── __init__.py  
│   ├── robot_coordinator.py # (unchanged)  
│   ├── robot_controller.py  # (unchanged)  
│   ├── multi_robot_slam.py  # (retained but NOT launched)  
│   └─ map_merger.py         # NEW: dynamic multi-map fuser  
├─ launch/  
│   └─ multirobot_launch.py  # MOD: call slam_toolbox + EKF +  
map_merger  
├─ └─ slam_merge.launch.py   # NEW: standalone launch for slam+merge  
└─ config/  
    ├── slam_common.yaml     # NEW: common params for slam_toolbox  
    └─ ekf_common.yaml       # NEW: common params for  
robot_localization  
└─ └─ map_merger.yaml        # NEW: global grid & alignment params
```

1) package.xml (MOD)

```
<?xml version="1.0"?>  
<package format="3">  
  <name>multirobot_nav</name>  
  <version>0.1.0</version>  
  <description>Multi-robot mapping & coordination with slam_toolbox + dynamic  
map merge</description>  
  
  <maintainer email="you@example.com">Your Name</maintainer>  
  <license>MIT</license>  
  
  <!-- Core -->  
  <depend>rclpy</depend>  
  <depend>std_msgs</depend>  
  <depend>geometry_msgs</depend>  
  <depend>nav_msgs</depend>  
  <depend>sensor_msgs</depend>  
  <depend>tf2_ros</depend>
```

```

<depend>tf2_geometry_msgs</depend>

<!-- External executables launched by this package -->
<exec_depend>slam_toolbox</exec_depend>
<exec_depend>robot_localization</exec_depend>

<!-- Python runtime helpers -->
<exec_depend>python3-numpy</exec_depend>
<exec_depend>python3-yaml</exec_depend>

<!-- Test/Lint (optional) -->
<test_depend>ament_flake8</test_depend>
<test_depend>ament_pep257</test_depend>
<test_depend>ament_pytest</test_depend>

<export>
  <build_type>ament_python</build_type>
</export>
</package>

```

2) `setup.py` (MOD)

```

from setuptools import setup

package_name = 'multirobot_nav'

setup(
    name=package_name,
    version='0.1.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages', ['resource/' +
package_name]),
        ('share/' + package_name, ['package.xml']),
        ('share/' + package_name + '/launch', [
            'launch/multirobot_launch.py',
            'launch/slam_merge.launch.py',
        ]),
        ('share/' + package_name + '/config', [
            'config/slam_common.yaml',
            'config/ekf_common.yaml',
            'config/map_merger.yaml',
        ])
    ],

```

```

install_requires=['setuptools', 'numpy', 'PyYAML'],
zip_safe=True,
maintainer='Your Name',
maintainer_email='you@example.com',
description='Multi-robot mapping with slam_toolbox and dynamic map merging',
license='MIT',
entry_points={
    'console_scripts': [
        'robot_coordinator = multirobot_nav.robot_coordinator:main',
        'robot_controller = multirobot_nav.robot_controller:main',
        # old custom SLAM disabled, but kept:
        'multi_robot_slam = multirobot_nav.multi_robot_slam:main',
        # NEW dynamic map merger
        'map_merger = multirobot_nav.map_merger:main',
    ],
},
)

```

If your existing `setup.py` already has entries, just append the `map_merger` line and add the config + launch files to `data_files`.

3) `config/slam_common.yaml` (NEW)

```

slam_toolbox:
  ros__parameters:
    # Generic tuning (overridden per-robot for frames & scan topic via launch)
    resolution: 0.05          # m/cell
    minimum_time_interval: 0.0
    max_laser_range: 12.0
    use_scan_matching: true
    loop_search_maximum_distance: 10.0
    do_loop_closing: true
    use_slam_toolbox_pose_graph: true

    mode: "mapping"           # switch to "localization" when using a saved map
    publish_tf: true          # publishes <map> -> <odom>

    # Frames/scan_topic will be overridden from launch for each robot:
    # base_frame, odom_frame, map_frame, scan_topic

```

4) config/ekf_common.yaml (NEW)

```
# Common EKF settings; frames & odom topic overridden per robot from launch
ekf_filter_node:
  ros__parameters:
    two_d_mode: true
    publish_tf: true
    sensor_timeout: 0.1

    # Default fusion mask for nav_msgs/Odometry (x,y,yaw)
    odom0_config: [ true,  true,  false,
                    false, false, true,
                    false, false, false,
                    false, false, false,
                    false, false, false ]

    odom0_differential: false
    odom0_relative: false
```

5) config/map_merger.yaml (NEW)

```
map_merger:
  ros__parameters:
    global_frame: "map"           # shared world frame for the merged map
    resolution: 0.05              # merged grid resolution (m/cell)
    width: 800                    # cells -> 40 m if res=0.05
    height: 800                   # cells
    origin_x: -20.0               # meters
    origin_y: -20.0               # meters

    # Namespaces of robots we will merge
    robots: ["robot1", "robot2"]

    # Alignment transforms (robot_map -> global map) in meters/radians.
    # You can tune at runtime with: ros2 param set /map_merger alignment.robot1
    "x y yaw"
    alignment:
      robot1: "0.0 0.0 0.0"
      robot2: "2.0 0.0 0.0"

    # Publishing
    publish_rate_hz: 2.0
```

The `alignment` values seed the relative pose of each robot's local map frame (`<ns>/map`) w.r.t. the global `map`. You can change them while running with ROS 2 param set commands; the merger updates live.

6) `multirobot_nav/map_merger.py` (NEW)

```
#!/usr/bin/env python3
import math
import numpy as np
from typing import Dict, Tuple

import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, DurabilityPolicy, ReliabilityPolicy,
HistoryPolicy

from nav_msgs.msg import OccupancyGrid, MapMetaData
from geometry_msgs.msg import TransformStamped

class MapMerger(Node):
    def __init__(self):
        super().__init__('map_merger')

        # --- Params ---
        self.declare_parameter('global_frame', 'map')
        self.declare_parameter('resolution', 0.05)
        self.declare_parameter('width', 800)
        self.declare_parameter('height', 800)
        self.declare_parameter('origin_x', -20.0)
        self.declare_parameter('origin_y', -20.0)
        self.declare_parameter('robots', ['robot1', 'robot2'])
        self.declare_parameter('alignment', {})
        self.declare_parameter('publish_rate_hz', 2.0)

        self.global_frame =
self.get_parameter('global_frame').get_parameter_value().string_value
        self.res = float(self.get_parameter('resolution').value)
        self.W = int(self.get_parameter('width').value)
        self.H = int(self.get_parameter('height').value)
        self.origin_x = float(self.get_parameter('origin_x').value)
        self.origin_y = float(self.get_parameter('origin_y').value)
        self.robot_ns = [str(s) for s in self.get_parameter('robots').value]
        self.publish_rate = float(self.get_parameter('publish_rate_hz').value)
```

```

# alignment dict of ns -> "x y yaw"
self.align_spec: Dict[str, str] = self.get_parameter('alignment').value
self.align_T: Dict[str, np.ndarray] = {}

# QoS to receive latched maps reliably and future updates
map_qos = QoSProfile(
    reliability=ReliabilityPolicy.RELIABLE,
    durability=DurabilityPolicy.TRANSIENT_LOCAL,
    history=HistoryPolicy.KEEP_LAST,
    depth=1,
)

# Subscriptions per robot
self.subs = {}
self.latest_maps: Dict[str, Tuple[MapMetaData, np.ndarray]] = {}
for ns in self.robot_ns:
    topic = f'/{ns}/map'
    self.subs[ns] = self.create_subscription(
        OccupancyGrid, topic,
        lambda msg, ns=ns: self._on_map(ns, msg), map_qos
    )
    # init align transform
    self.align_T[ns] = self._spec_to_T(self.align_spec.get(ns, '0 0 0'))

# Output (latched publish)
out_qos = QoSProfile(
    reliability=ReliabilityPolicy.RELIABLE,
    durability=DurabilityPolicy.TRANSIENT_LOCAL,
    history=HistoryPolicy.KEEP_LAST,
    depth=1,
)
self.pub = self.create_publisher(OccupancyGrid, '/map', out_qos)

# Pre-allocate global grid
self.global_grid = np.full((self.H, self.W), -1, dtype=np.int8)
self.global_info = MapMetaData()
self.global_info.resolution = self.res
self.global_info.width = self.W
self.global_info.height = self.H
self.global_info.origin.position.x = self.origin_x
self.global_info.origin.position.y = self.origin_y
self.global_info.origin.position.z = 0.0
self.global_info.origin.orientation.w = 1.0

# Publish timer
self.create_timer(1.0 / max(self.publish_rate, 0.1), self._publish)

# Dynamic parameter updates (alignments)

```

```

self.add_on_set_parameters_callback(self._on_param_set)

self.get_logger().info(
    f"map_merger ready. robots={self.robot_ns}, global {self.W}
x{self.H} @ {self.res} m/cell"
)

# --- Helpers ---
def _spec_to_T(self, spec: str) -> np.ndarray:
    try:
        x, y, yaw = [float(v) for v in spec.replace(',', ' ').split()[:3]]
    except Exception:
        x, y, yaw = 0.0, 0.0, 0.0
    c, s = math.cos(yaw), math.sin(yaw)
    T = np.array([[c, -s, x],
                  [s,  c, y],
                  [0,  0, 1]], dtype=np.float64)
    return T

def _on_param_set(self, params):
    changed = False
    for p in params:
        # Expected names like: alignment.robot1
        if p.name.startswith('alignment.'):
            ns = p.name.split('.', 1)[1]
            self.align_T[ns] = self._spec_to_T(p.value)
            self.get_logger().info(f"Updated alignment {ns}: {p.value}")
            changed = True
    from rclpy.parameter import SetParametersResult
    return SetParametersResult(successful=True)

def _on_map(self, ns: str, msg: OccupancyGrid):
    # Convert to numpy
    info = msg.info
    data = np.array(msg.data, dtype=np.int16).reshape((info.height,
info.width))
    self.latest_maps[ns] = (info, data)
    # Update global grid immediately on each incoming map
    self._fuse_all()

def _world_to_global_ij(self, x: np.ndarray, y: np.ndarray) ->
Tuple[np.ndarray, np.ndarray]:
    gi = np.floor((y - self.origin_y) / self.res).astype(np.int64)
    gj = np.floor((x - self.origin_x) / self.res).astype(np.int64)
    return gi, gj

def _fuse_map(self, ns: str, info: MapMetaData, grid: np.ndarray, T_align:
np.ndarray, canvas: np.ndarray):

```

```

# Coordinates of cell centers in local map frame
h, w = grid.shape
# mask of known cells
known = grid >= 0
if not np.any(known):
    return

yi, xi = np.nonzero(known)
vals = grid[yi, xi]

# local metric coords
x_local = info.origin.position.x + (xi + 0.5) * info.resolution
y_local = info.origin.position.y + (yi + 0.5) * info.resolution

# to global via alignment transform
ones = np.ones_like(x_local)
P = np.vstack((x_local, y_local, ones))
Pg = T_align @ P
xg, yg = Pg[0, :], Pg[1, :]

# to global grid indices
gi, gj = self._world_to_global_ij(xg, yg)

# filter in-bounds
inside = (gi >= 0) & (gi < self.H) & (gj >= 0) & (gj < self.W)
gi, gj, vals = gi[inside], gj[inside], vals[inside]

# fuse: prefer occupied over free; else take max known
# unknown=-1, free=0, occ=100 (slam_toolbox convention)
flat_idx = gi * self.W + gj
# Use np.maximum but preserve 100 if either is 100
cur = canvas.flatten()
cur_vals = cur[flat_idx]
# unknown -> assign
replace_unknown = (cur_vals < 0)
cur_vals[replace_unknown] = vals[replace_unknown]
# otherwise take max
cur_vals[~replace_unknown] = np.maximum(cur_vals[~replace_unknown],
vals[~replace_unknown])
cur[flat_idx] = cur_vals
canvas[:] = cur.reshape((self.H, self.W))

def _fuse_all(self):
    # reset canvas and re-compose every time (simple & robust)
    canvas = np.full((self.H, self.W), -1, dtype=np.int8)
    for ns, (info, grid) in self.latest_maps.items():
        self._fuse_map(ns, info, grid, self.align_T.get(ns, np.eye(3)),
canvas)

```



```

        self.global_grid = canvas

    def _publish(self):
        msg = OccupancyGrid()
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = self.global_frame
        msg.info = self.global_info
        msg.data = list(self.global_grid.flatten().astype(np.int8))
        self.pub.publish(msg)

def main():
    rclpy.init()
    node = MapMerger()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    rclpy.shutdown()

```

Usage tips - Inspect the merged `/map` in RViz. If the two local maps don't overlap nicely, tune the alignment live, e.g.:

```
ros2 param set /map_merger alignment.robot2 "1.5 0.3 0.08"
```

- The node recomposes the whole canvas at each update for robustness. If needed, you can optimize to only redraw changed tiles.

7) `launch/slam_merge.launch.py` (NEW)

```

#!/usr/bin/env python3
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, GroupAction
from launch.substitutions import ImportLaunchConfiguration
from launch_ros.actions import Node, PushRosNamespace
from launch_ros.parameter_descriptions import ParameterFile

def per_robot_group(ns: str, slam_yaml: str, ekf_yaml: str):
    # Slam frames & scan topic are overridden here to avoid N YAML copies
    slam_overrides = {
        'base_frame': f'{ns}/base_link',

```

```

        'odom_frame': f'{ns}/odom',
        'map_frame': f'{ns}/map',
        'scan_topic': f'/{ns}/scan', # slam_toolbox expects absolute
    }

    ekf_overrides = {
        'base_link_frame': f'{ns}/base_link',
        'odom_frame': f'{ns}/odom',
        'world_frame': f'{ns}/odom',
        'odom0': f'/{ns}/odom',
    }

    return GroupAction([
        PushRosNamespace(ns),
        Node(
            package='robot_localization', executable='ekf_node',
            name='ekf_filter_node',
            parameters=[ekf_yaml, ekf_overrides],
            output='screen'
        ),
        Node(
            package='slam_toolbox', executable='async_slam_toolbox_node',
            name='slam_toolbox',
            parameters=[slam_yaml, slam_overrides],
            output='screen'
        ),
    ])

def generate_launch_description():
    robots_arg = DeclareLaunchArgument(
        'robots', default_value='robot1,robot2',
        description='Comma-separated robot namespaces'
    )

    slam_yaml = ParameterFile('config/slam_common.yaml', allow_substs=True)
    ekf_yaml = ParameterFile('config/ekf_common.yaml', allow_substs=True)

    robots_cfg = LaunchConfiguration('robots')

    # At runtime we parse the list by a tiny helper node-free trick
    robots = [s.strip() for s in str(robots_cfg.perform(None) or
        'robot1,robot2').split(',')]

    groups = [per_robot_group(ns, slam_yaml, ekf_yaml) for ns in robots]

    # Global map merger (no namespace)
    merger = Node(

```

```

        package='multirobot_nav', executable='map_merger', name='map_merger',
        parameters=['config/map_merger.yaml'],
        output='screen'
    )

    return LaunchDescription([robots_arg] + groups + [merger])

```

Note: some launch frontends don't evaluate `LaunchConfiguration.perform(None)` at generation time. If your environment complains, hardcode the list in early iterations or replace with `OpaqueFunction` to parse at runtime.

8) `launch/multirobot_launch.py` (MOD)

Replace your previous SLAM block and any static `map->odom` TFs with the following pattern (keep your controllers/coordinator nodes as-is):

```

#!/usr/bin/env python3
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    pkg_share = get_package_share_directory('multirobot_nav')

    # Include the slam+merge pipeline defined above
    slam_merge = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(pkg_share, 'launch',
            'slam_merge.launch.py')),
        launch_arguments={'robots': 'robot1,robot2'}.items()
    )

    # Your existing controllers/coordinator
    coordinator = Node(
        package='multirobot_nav', executable='robot_coordinator',
        name='robot_coordinator',
        parameters=['config/map_merger.yaml'], # so it knows the global frame/
        res if needed
        output='screen'
    )

    # Example: controllers per robot (if not already launched elsewhere)
    ctrl1 = Node(

```

```

        package='multirobot_nav', executable='robot_controller',
name='controller',
        namespace='robot1', parameters=['config/robot_params.yaml'],
output='screen')

    ctrl2 = Node(
        package='multirobot_nav', executable='robot_controller',
name='controller',
        namespace='robot2', parameters=['config/robot_params.yaml'],
output='screen')

    return LaunchDescription([slam_merge, coordinator, ctrl1, ctrl2])

```

How to run

```

# 1) Build the package
cd ~/multirobot_ws
colcon build --symlink-install
source install/setup.bash

# 2) Launch SLAM + EKF per robot and the dynamic map merger
ros2 launch multirobot_nav slam_merge.launch.py robots:=robot1,robot2

# (or) use your main launch which includes the above
ros2 launch multirobot_nav multirobot_launch.py

# 3) Tune alignments live until the merged map looks right in RViz
ros2 param set /map_merger alignment.robot2 "1.5 0.3 0.08"

# 4) Visualize
rviz2 # Fixed Frame: map, display /map (merged), /robot1/map, /robot2/map, TF

```

Notes - `slam_toolbox` publishes `<ns>/map -> <ns>/odom` dynamically; the EKF publishes `<ns>/odom -> <ns>/base_link`. Do not add static TFs for these links. - The merger recomputes `/map` on every incoming local map update and at a steady publish rate, so the global map is **fully dynamic**. - For larger teams or better alignment, drop landmarks (AprilTags) and set alignment from tag-based localization, or add a simple scan-matching refinement step before fusion.

Final files (based on your current repo & agents on ports 8888/8889)

These are complete drop-in files. Keep your existing micro-ROS firmware publishing to `/robot1/*` and `/robot2/*`. The launch keeps your two agents.

`package.xml` (replace)

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypelevel="1"?>
<package format="3">
  <name>multirobot_nav</name>
  <version>0.1.0</version>
  <description>Multi-robot navigation with slam_toolbox and dynamic map merge</
description>
  <maintainer email="sansikasuhan5@gmail.com">Platinum-Saber</maintainer>
  <license>MIT</license>

  <!-- Core -->
  <depend>rclpy</depend>
  <depend>sensor_msgs</depend>
  <depend>geometry_msgs</depend>
  <depend>nav_msgs</depend>
  <depend>tf2_ros</depend>
  <depend>tf2_geometry_msgs</depend>
  <depend>laser_geometry</depend>

  <!-- External executables we launch -->
  <exec_depend>slam_toolbox</exec_depend>
  <exec_depend>robot_localization</exec_depend>

  <!-- Python runtime helpers for map_merger -->
  <exec_depend>python3-numpy</exec_depend>
  <exec_depend>python3-yaml</exec_depend>

  <!-- Test/Lint (optional) -->
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>ament_pytest</test_depend>

  <export>
    <build_type>ament_python</build_type>
```

```
</export>
</package>
```

setup.py (replace)

```
from setuptools import setup, find_packages
import os
from glob import glob

package_name = 'multirobot_nav'

setup(
    name=package_name,
    version='0.1.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages', ['resource/' +
package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share', package_name, 'launch'), glob('launch/*.py')),
        (os.path.join('share', package_name, 'config'), glob('config/*.yaml')),
    ],
    install_requires=['setuptools', 'numpy', 'PyYAML'],
    zip_safe=True,
    maintainer='Platinum-Saber',
    maintainer_email='sansikasuhan5@gmail.com',
    description='Multi-robot navigation with slam_toolbox and dynamic map
merge',
    license='MIT',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'robot_coordinator = multirobot_nav.robot_coordinator:main',
            'robot_controller = multirobot_nav.robot_controller:main',
            'multi_robot_slam = multirobot_nav.multi_robot_slam:main', # kept
for reference, not launched
            'map_merger = multirobot_nav.map_merger:main', # NEW
        ],
    },
)
```

config/slam_common.yaml (new)

```
slam_toolbox:
  ros__parameters:
    # These are common defaults; frames and scan_topic are overridden per-robot
    in launch
    resolution: 0.05
    minimum_time_interval: 0.0
    max_laser_range: 12.0
    use_scan_matching: true
    do_loop_closing: true
    loop_search_maximum_distance: 10.0
    use_slam_toolbox_pose_graph: true
    mode: "mapping"      # change to "localization" when using a saved map
    publish_tf: true     # publishes <ns>/map -> <ns>/odom
    # base_frame / odom_frame / map_frame / scan_topic will be injected from
    launch
```

config/ekf_common.yaml (new)

```
ekf_filter_node:
  ros__parameters:
    two_d_mode: true
    publish_tf: true
    sensor_timeout: 0.1

    # Fuse x, y, yaw from wheel odometry
    odom0_config: [ true,  true,  false,
                    false, false, true,
                    false, false, false,
                    false, false, false,
                    false, false, false ]
    odom0_differential: false
    odom0_relative: false
    # base_link_frame / odom_frame / world_frame / odom0 will be injected from
    launch
```

config/map_merger.yaml (new)

```
map_merger:
  ros__parameters:
    global_frame: "map"
    resolution: 0.05
```

```

width: 800          # 40m @ 5cm
height: 800
origin_x: -20.0
origin_y: -20.0

robots: ["robot1", "robot2"]

# Initial guesses for local <ns>/map -> global /map (x y yaw)
alignment:
  robot1: "0.0 0.0 0.0"
  robot2: "2.0 0.0 0.0"

publish_rate_hz: 2.0

```

multirobot_nav/map_merger.py (new)

```

#!/usr/bin/env python3
import math
import numpy as np
from typing import Dict, Tuple

import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, DurabilityPolicy, ReliabilityPolicy,
HistoryPolicy

from nav_msgs.msg import OccupancyGrid, MapMetaData

class MapMerger(Node):
    def __init__(self):
        super().__init__('map_merger')

        # Params
        self.declare_parameter('global_frame', 'map')
        self.declare_parameter('resolution', 0.05)
        self.declare_parameter('width', 800)
        self.declare_parameter('height', 800)
        self.declare_parameter('origin_x', -20.0)
        self.declare_parameter('origin_y', -20.0)
        self.declare_parameter('robots', ['robot1', 'robot2'])
        self.declare_parameter('alignment', {})
        self.declare_parameter('publish_rate_hz', 2.0)

        self.global_frame =
self.get_parameter('global_frame').get_parameter_value().string_value

```



```

self.res = float(self.get_parameter('resolution').value)
self.W = int(self.get_parameter('width').value)
self.H = int(self.get_parameter('height').value)
self.origin_x = float(self.get_parameter('origin_x').value)
self.origin_y = float(self.get_parameter('origin_y').value)
self.robot_ns = [str(s) for s in self.get_parameter('robots').value]
self.publish_rate = float(self.get_parameter('publish_rate_hz').value)

# alignment dictionary: ns -> "x y yaw"
self.align_spec: Dict[str, str] = self.get_parameter('alignment').value
self.align_T: Dict[str, np.ndarray] = {ns:
self._spec_to_T(self.align_spec.get(ns, '0 0 0')) for ns in self.robot_ns}

# QoS to get latched last map and future updates
map_qos = QoSProfile(
    reliability=ReliabilityPolicy.RELIABLE,
    durability=DurabilityPolicy.TRANSIENT_LOCAL,
    history=HistoryPolicy.KEEP_LAST,
    depth=1,
)

# Subscriptions per robot
self.subs = {}
self.latest_maps: Dict[str, Tuple[MapMetaData, np.ndarray]] = {}
for ns in self.robot_ns:
    topic = f'/{ns}/map'
    self.subs[ns] = self.create_subscription(
        OccupancyGrid, topic,
        lambda msg, ns=ns: self._on_map(ns, msg), map_qos
    )

# Output publisher (latched)
out_qos = QoSProfile(
    reliability=ReliabilityPolicy.RELIABLE,
    durability=DurabilityPolicy.TRANSIENT_LOCAL,
    history=HistoryPolicy.KEEP_LAST,
    depth=1,
)
self.pub = self.create_publisher(OccupancyGrid, '/map', out_qos)

# Pre-allocate global grid
self.global_grid = np.full((self.H, self.W), -1, dtype=np.int8)
self.global_info = MapMetaData()
self.global_info.resolution = self.res
self.global_info.width = self.W
self.global_info.height = self.H
self.global_info.origin.position.x = self.origin_x
self.global_info.origin.position.y = self.origin_y

```

```

self.global_info.origin.position.z = 0.0
self.global_info.origin.orientation.w = 1.0

# Timed publisher
self.create_timer(1.0 / max(self.publish_rate, 0.1), self._publish)

# Allow live alignment updates: ros2 param set /map_merger
alignment.robot2 "dx dy yaw"
self.add_on_set_parameters_callback(self._on_param_set)

self.get_logger().info(
    f"map_merger ready. robots={self.robot_ns}, global {self.W}
x{self.H} @ {self.res} m/cell"
)

# ----- helpers -----
def _spec_to_T(self, spec: str) -> np.ndarray:
    try:
        x, y, yaw = [float(v) for v in spec.replace(',', ' ').split()[:3]]
    except Exception:
        x, y, yaw = 0.0, 0.0, 0.0
    c, s = math.cos(yaw), math.sin(yaw)
    return np.array([[c, -s, x],
                    [s, c, y],
                    [0, 0, 1]], dtype=np.float64)

def _on_param_set(self, params):
    from rclpy.parameter import SetParametersResult
    for p in params:
        if p.name.startswith('alignment.'):
            ns = p.name.split('.', 1)[1]
            self.align_T[ns] = self._spec_to_T(p.value)
            self.get_logger().info(f"Updated alignment {ns}: {p.value}")
    return SetParametersResult(successful=True)

def _on_map(self, ns: str, msg: OccupancyGrid):
    info = msg.info
    grid = np.array(msg.data, dtype=np.int16).reshape((info.height,
info.width))
    self.latest_maps[ns] = (info, grid)
    self._recompose()

def _world_to_global_ij(self, x: np.ndarray, y: np.ndarray):
    gi = np.floor((y - self.origin_y) / self.res).astype(np.int64)
    gj = np.floor((x - self.origin_x) / self.res).astype(np.int64)
    return gi, gj

def _fuse(self, ns: str, info: MapMetaData, grid: np.ndarray, T_align:

```

```

np.ndarray, canvas: np.ndarray):
    h, w = grid.shape
    known = grid >= 0
    if not np.any(known):
        return
    yi, xi = np.nonzero(known)
    vals = grid[yi, xi]

    # local metric coords (cell centers)
    x_local = info.origin.position.x + (xi + 0.5) * info.resolution
    y_local = info.origin.position.y + (yi + 0.5) * info.resolution

    ones = np.ones_like(x_local)
    P = np.vstack((x_local, y_local, ones))
    Pg = T_align @ P
    xg, yg = Pg[0, :], Pg[1, :]

    gi, gj = self._world_to_global_ij(xg, yg)
    inside = (gi >= 0) & (gi < self.H) & (gj >= 0) & (gj < self.W)
    gi, gj, vals = gi[inside], gj[inside], vals[inside]

    flat = gi * self.W + gj
    cur = canvas.ravel()
    cur_vals = cur[flat]
    # unknown -> assign, else max
    unknown = (cur_vals < 0)
    cur_vals[unknown] = vals[unknown]
    cur_vals[~unknown] = np.maximum(cur_vals[~unknown], vals[~unknown])
    cur[flat] = cur_vals

def _recompose(self):
    canvas = np.full((self.H, self.W), -1, dtype=np.int8)
    for ns, (info, grid) in self.latest_maps.items():
        self._fuse(ns, info, grid, self.align_T.get(ns, np.eye(3)), canvas)
    self.global_grid = canvas

def _publish(self):
    msg = OccupancyGrid()
    msg.header.stamp = self.get_clock().now().to_msg()
    msg.header.frame_id = self.global_frame
    msg.info = self.global_info
    msg.data = list(self.global_grid.flatten().astype(np.int8))
    self.pub.publish(msg)

def main():
    rclpy.init()
    node = MapMerger()

```

```

try:
    rclpy.spin(node)
except KeyboardInterrupt:
    pass
node.destroy_node()
rclpy.shutdown()

```

launch/slam_merge.launch.py (new)

```

#!/usr/bin/env python3
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, GroupAction, OpaqueFunction
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node, PushRosNamespace
from launch_ros.parameter_descriptions import ParameterFile

def per_robot_group(ns: str, slam_yaml: ParameterFile, ekf_yaml: ParameterFile):
    slam_overrides = {
        'base_frame': f'{ns}/base_link',
        'odom_frame': f'{ns}/odom',
        'map_frame': f'{ns}/map',
        'scan_topic': f'/{ns}/scan', # absolute path
    }
    ekf_overrides = {
        'base_link_frame': f'{ns}/base_link',
        'odom_frame': f'{ns}/odom',
        'world_frame': f'{ns}/odom',
        'odom0': f'/{ns}/odom',
    }
    return GroupAction([
        PushRosNamespace(ns),
        Node(
            package='robot_localization', executable='ekf_node',
            name='ekf_filter_node',
            parameters=[ekf_yaml, ekf_overrides],
            output='screen'
        ),
        Node(
            package='slam_toolbox', executable='async_slam_toolbox_node',
            name='slam_toolbox',
            parameters=[slam_yaml, slam_overrides],
            output='screen'
        ),
    ])

```

```

def _expand_robots(context, *args, **kwargs):
    robots = LaunchConfiguration('robots').perform(context)
    robots = [s.strip() for s in robots.split(',') if s.strip()]

    slam_yaml = ParameterFile('config/slam_common.yaml', allow_substs=True)
    ekf_yaml = ParameterFile('config/ekf_common.yaml', allow_substs=True)

    actions = [per_robot_group(ns, slam_yaml, ekf_yaml) for ns in robots]

    # Global map merger (root ns)
    actions.append(Node(
        package='multirobot_nav', executable='map_merger', name='map_merger',
        parameters=['config/map_merger.yaml'], output='screen'))

    return actions

def generate_launch_description():
    return LaunchDescription([
        DeclareLaunchArgument('robots', default_value='robot1,robot2',
                               description='Comma-separated robot namespaces'),
        OpaqueFunction(function=_expand_robots),
    ])

```

launch/multi_robot_launch.py (replace)

```

#!/usr/bin/env python3
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    pkg_share = get_package_share_directory('multirobot_nav')

    # micro-ROS agents (match your firmware):
    agent_r1 = Node(
        package='micro_ros_agent', executable='micro_ros_agent',
        arguments=['udp4', '--port', '8888'], name='micro_ros_agent_robot1',
        output='screen')
    agent_r2 = Node(
        package='micro_ros_agent', executable='micro_ros_agent',

```

```

        arguments=['udp4', '--port', '8889'], name='micro_ros_agent_robot2',
        output='screen')

# Base -> LiDAR static TFs only (slam_toolbox & EKF will publish the dynamic
links)
    base_to_lidar_r1 = Node(
        package='tf2_ros', executable='static_transform_publisher',
        name='robot1_base_to_lidar',
        arguments=['0', '0', '0.10', '0', '0', '0', 'robot1/base_link',
'robot1/lidar_link'])
    base_to_lidar_r2 = Node(
        package='tf2_ros', executable='static_transform_publisher',
        name='robot2_base_to_lidar',
        arguments=['0', '0', '0.10', '0', '0', '0', 'robot2/base_link',
'robot2/lidar_link'])

    # Include SLAM + EKF per robot + dynamic map merger
    slam_merge = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(pkg_share, 'launch',
'slam_merge.launch.py')),
        launch_arguments={'robots': 'robot1,robot2'}.items()
    )

    # Your controllers & coordinator (unchanged)
    ctrl1 = Node(
        package='multirobot_nav', executable='robot_controller',
        name='controller',
        namespace='robot1', parameters=['config/robot_params.yaml'],
        output='screen')
    ctrl2 = Node(
        package='multirobot_nav', executable='robot_controller',
        name='controller',
        namespace='robot2', parameters=['config/robot_params.yaml'],
        output='screen')

    coordinator = Node(
        package='multirobot_nav', executable='robot_coordinator',
        name='robot_coordinator',
        parameters=['config/map_merger.yaml'], output='screen')

    return LaunchDescription([
        agent_r1, agent_r2,
        base_to_lidar_r1, base_to_lidar_r2,
        slam_merge,
        ctrl1, ctrl2,
        coordinator,
    ])

```

Run commands

```
cd ~/multirobot_ws
colcon build --symlink-install
source install/setup.bash

# Bring up agents + slam_toolbox + EKF + dynamic merger + controllers
ros2 launch multirobot_nav multirobot_launch.py

# Live tune alignment while watching RViz
ros2 param set /map_merger alignment.robot2 "1.5 0.3 0.08"
```

TF ownership recap - `slam_toolbox` (per robot) publishes `<ns>/map -> <ns>/odom`. - `robot_localization` EKF publishes `<ns>/odom -> <ns>/base_link`. - Keep only the static `base_link -> lidar_link` publishers you already had.